

1 Laden von Konstanten in Register

Anw-01 Kleine Konstanten können direkt im Maschinenwort untergebracht werden.

Anw-02 Registerinhalte (im Hex-Format) nach Ausführung von Anw-02:

Register R0				Register R1			
00	00	00	12	ff	ff	ff	80
31	16	15	0	31	16	15	0

Anw-03 Größere Konstante erfordern einen zusätzlichen Speicherplatz zum Speichern des Wertes. Dieser kann automatisch vom Assembler erzeugt werden (Gleichheitszeichen). Durch Auswertung des Disassemblers kann die Adresse des Speicherplatzes bestimmt werden:

Adresse der Konstanten:				Inhalt Register R2 nach Anw-03:			
08	00	20	30	12	34	56	78
31	16	15	0	31	16	15	0

Zum Zugriff auf den zusätzlichen Speicherplatz setzt der Assembler eine besondere Assembleranweisung ein. Geben Sie an, welche Anweisung die CPU tatsächlich ausführt:

r2, [pc, #60]

2 Zugriff auf Variable

Anw-04 Variable werden im Datenbereich des Prozessors untergebracht. Auf die Variablen kann nicht direkt zugegriffen werden, sondern es muss

Anw-05 zuvor deren Speicheradresse in ein Register geladen werden:

Adresse v. VariableA (Reg. R0 nach Anw-05)

20	00	00	0C
31	16	15	0

Inhalt Register R1 nach Anw-05:

00	00	12	34
31	16	15	0

Anzahl von Anw-05 geladener Bits:

16

Anw-06 Der Assembler überprüft nicht, ob der Lade/Speicherbefehl mit der Definition der Variable übereinstimmt. Anw-06 lädt 32 Bits, obwohl VariableA nur 16 Bit umfasst:

Inhalt Register R2 nach Anw-06:

47	11	12	34
31	16	15	0

Anzahl von Anw-06 geladener Bits:

32

Anw-07 Die Adresse einer Variable kann auch als Differenz zu einer Basis angegeben werden:

Adresse v. VariableC:

20	00	00	10
31	16	15	0

Inhalt von VariableC nach Anw-07:

34	12	11	47
31	16	15	0

Anzahl von Anw-07 gespeicherter Bits:

32

3 Zugriff auf Felder

Die Elemente eines Feldes sind alle vom gleichen Typ und werden der Reihe nach im Speicher angelegt.

	Anfangsadresse:	Anzahl Elemente:	Anzahl Bytes pro Element:										
MeinHalbwortFeld:	<table><tr><td>20</td><td>00</td><td>00</td><td>14</td></tr><tr><td>31</td><td>16</td><td>15</td><td>0</td></tr></table>	20	00	00	14	31	16	15	0	<table><tr><td>6</td></tr></table>	6	<table><tr><td>2</td></tr></table>	2
20	00	00	14										
31	16	15	0										
6													
2													
MeinWortFeld:	<table><tr><td>20</td><td>00</td><td>00</td><td>20</td></tr><tr><td>31</td><td>16</td><td>15</td><td>0</td></tr></table>	20	00	00	20	31	16	15	0	<table><tr><td>6</td></tr></table>	6	<table><tr><td>4</td></tr></table>	4
20	00	00	20										
31	16	15	0										
6													
4													
MeinTextFeld:	<table><tr><td>20</td><td>00</td><td>00</td><td>38</td></tr><tr><td>31</td><td>16</td><td>15</td><td>0</td></tr></table>	20	00	00	38	31	16	15	0	<table><tr><td>9</td></tr></table>	9	<table><tr><td>1</td></tr></table>	1
20	00	00	38										
31	16	15	0										
9													
1													

Anw-08 Der Zugriff auf ein bestimmtes Element kann durch das Laden der Anfangsadresse in ein Register und die Angabe der Adressdifferenz für
Anw-09 das jeweilige Element erfolgen.

Anw-10

Register R1 nach Anw-09:

00

00

00

22

31

16

15

0

Register R2 nach Anw-10:

00

00

00

3E

31

16

15

0

Anw-11 Die Adressdifferenz kann auch über ein Register angegeben werden. Sie kann somit während des Programmlaufs verändert werden:
Anw-12

Register R4 nach Anw-12:				Nr. (beginnend mit 0) des von Anw-12 gelesenen Elements:	
00	00	00	45	5	
31	16	15	0		

Anw-13 Die Basisadresse kann automatisch nach Zugriff auf ein Element erhöht werden.

Register R0 nach Anw-13				Register R5 nach Anw-13:				Nr. (beginnend mit 0) des von Anw-13 gelesenen Elements:	
20	00	00	16	00	00	00	3E	1	
31	16	15	0	31	16	15	0		

Anw-14

Register R0 nach Anw-14

20

00

00

18

31

16

15

0

Register R6 nach Anw-14:

00

00

FF

CC

31

16

15

0

Nr. (beginnend mit 0) des
von Anw-14 gelesenen Elements:

2

Anw-15

Register R0 nach Anw-15

Adresse des von Anw-15 überschriebenen Elements:

Nr. (beginnend mit 0) des
von Anw-15 überschriebenen Elements:

20	00	00	1A	00	00	00	00
31	16	15	0	31	16	15	0

2

4 Addition und Subtraktion von vorzeichenlosen und vorzeichenbehafteten Integer-Werten

Arithmetik-Befehle haben 3 Operanden. Tragen Sie in das nachfolgende Schema die beteiligten Werte in binärer Form ein. Wandeln Sie die binäre Darstellungen um in vorzeichenbehaftete und vorzeichenlose Dezimalzahlen. Überprüfen Sie die Rechnung. Stimmen die Ergebnisse mit den Erwartungen überein?

Anw-16

Anw-17

Anw-18

Anw-19

		dezimale Interpretation der Bitmuster																															
		signed																unsigned															
Reg. R1:	0001001000011010001010110011110000	305419896																305419896															
Reg. R2:	100111011100010100101100110000110	-1647683194																2647284102															
Reg. R3:	10101111111111101010111111111110	-1342263298																2952703998															
		Ergebnis:																Ergebnis:															
Kreuzen Sie die gesetzten Bits des Statusregisters an:		☒ Ok ☐ Nicht Ok																☒ Ok ☐ Nicht Ok															

Anw-20

Anw-21

Anw-22

		dezimale Interpretation der Bitmuster																															
		signed																unsigned															
Reg. R4:	11001100000000000000000000000000	-872415232																3422552064															
Reg. R5:	01001110000000000000000000000000	1308622848																1308622848															
Reg. R6:	01111110000000000000000000000000	2113929216																2113929216															
		Ergebnis:																Ergebnis:															
Kreuzen Sie die gesetzten Bits des Statusregisters an:		☐ Ok ☒ Nicht Ok																☐ Ok ☒ Nicht Ok															

Anw-23

Anw-24

Anw-25

0x27000000

0x45000000

0xe2000000

		dezimale Interpretation der Bitmuster																															
		signed																unsigned															
Reg. R7:	00100111000000000000000000000000	654311424																654311424															
Reg. R8:	01000101000000000000000000000000	1157627904																1157627904															
Reg. R9:	11100010000000000000000000000000	-503316480																3791650816															
		Ergebnis:																Ergebnis:															
Kreuzen Sie die gesetzten Bits des Statusregisters an:		☒ Ok ☐ Nicht Ok																☐ Ok ☒ Nicht Ok															